



PROJECT-BASED LEARNING

PHASE-I EVALUATION

Smart Playlist Shuffler: A Non-Repeating Audio Engine

Team ID: DSCPP-III-2026-T142

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026-27

Team & Mentor Details



Team Information

Team ID: *DSCPP-III-2026-T142*
Semester: *3rd*
Section: *C, ML3, ML2, H*
Domain: *Data Structures and Programming in C++*

Team Members

1. *Bhuvi Singhal – 2027720 (Leader)*
2. *Akshat Joshi – 2028632*
3. *Akshank Chauhan – 2028630*
4. *Tansi Singh – 2028216*

Mentor

Ms. Nishtha Bhatt

Interactions completed: _____

Problem Statement & Motivation

Problem Statement

Standard media players rely on basic pseudo-random functions like `rand()` that pick tracks without playback memory. Due to probability clustering, a small subset of songs repeats prematurely while others remain unplayed before the cycle restarts.

Motivation

- ▶ *Eliminate track clustering and early repetition in local media playback.*
- ▶ *Build a state-aware engine that actively tracks song playback history.*
- ▶ *Solve a practical systems problem by building custom C++ data structures without STL.*

Target Users / Stakeholders

- ▶ *Primary: Listeners who want a truly non-repeating local .wav audio experience.*
- ▶ *Secondary: Students and developers studying fair $O(n)$ shuffle mechanics.*
- ▶ *Benefit: A mathematically fair, guaranteed zero-repeat playback sequence.*

Project Objectives

1. *Guarantee a zero-repeat playback cycle by actively tracking played song IDs.*
2. *Implement the Fisher-Yates algorithm to generate an unbiased $O(n)$ track permutation.*
3. *Build all data structures (CDLL, Hash Set, Stack) manually from scratch without using STL.*
4. *Manage heap memory directly using custom destructors to guarantee zero memory leaks.*
5. *Interface with the Win32 Multimedia API to process .wav audio asynchronously in the background.*

Key Objective: *Deliver a fair randomization engine that strictly blocks songs from playing twice in a single traversal.*

Proposed Approach

1. *Input: Parse track data from playlist.txt.*
2. *Storage: Maintain cyclic playlist in a custom CDLL.*
3. *Processing: Permute song array via Fisher-Yates shuffle.*
4. *Tracking: Log IDs in Hash Set; store skips in LIFO Stack.*
5. *Output: Non-blocking audio playback via Win32 API.*

Key Features

- ▶ *Continuous cyclic traversal where the final track links back to the start*
- ▶ *Unbiased shuffle permutation with strictly zero early repetition*
- ▶ *Instant $O(1)$ duplicate verification backed by an open-addressed Hash Set*
- ▶ *Immediate skip-undo support powered by a transactional LIFO Stack*
- ▶ *Direct heap pointer management with verified zero memory leaks*

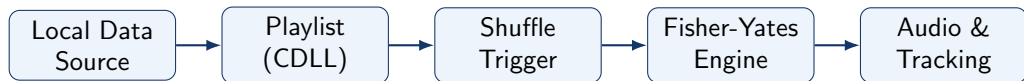
This approach replaces blind random guessing with a structured, state-aware playback queue.

Integration of PBL Course Concepts



Course	Concepts Applied	Project Module
Data Structures in C++	<i>Circular Doubly Linked Lists, Open-addressing Hash Sets, LIFO Stacks</i>	<i>Core Data Structures</i>
OOPs with C++	<i>Encapsulation, Dynamic Memory, Custom Constructors/Destructors</i>	<i>Container Classes</i>
Operating Systems	<i>Background thread offloading via Win32 SND_ASYNC flag</i>	<i>Audio Engine Pipeline</i>
Data Storage	<i>Sequential flat file stream parsing mapped to heap memory</i>	<i>File I/O Module</i>

Requirement: Demonstrate meaningful integration of both subjects in **one** project.



Major Components

- ▶ *Circular Doubly Linked List for infinite bidirectional traversal*
- ▶ *Fisher-Yates array permutation engine for unbiased $O(n)$ shuffling*
- ▶ *Custom open-addressing Hash Set for $O(1)$ duplicate track prevention*
- ▶ *LIFO History Stack recording played states for instant skip-undo*

Data / Control Flow

- ▶ *Program parses playlist.txt and dynamically constructs the CDLL*
- ▶ *User trigger copies list to array and executes the shuffle algorithm*
- ▶ *Hash Set blocks repeated IDs; completed songs push to LIFO Stack*
- ▶ *Win32 PlaySound streams .wav audio asynchronously on a secondary thread*

Technology Stack

- ▶ Programming: *Standard C++ (Raw pointers, manual memory, zero STL)*
- ▶ Audio Engine: *Win32 Multimedia API (mmsystem.h with SND_ASYNC)*
- ▶ Data Structures: *Custom CDLL, Open-addressing Hash Set, LIFO Stack*
- ▶ Compiler: *GCC (MinGW) or MSVC*
- ▶ Database: *Local Flat File Database (playlist.txt)*

Development Methodology

1. *Architect custom node topologies and bidirectional pointer layouts.*
2. *Implement custom CDLL, Hash Set, and Stack classes without STL.*
3. *Integrate Fisher-Yates shuffle and linear collision probing.*
4. *Bind Win32 async audio execution to the terminal command interface.*
5. *Profile destructors and memory logs to eliminate heap leaks.*

We kept the stack simple to focus entirely on writing strong logic and pointer management.

Initial Progress & Team Contribution



Progress Achieved

- ▶ *Formulated mathematical logic and swap flow for Fisher-Yates shuffle.*
- ▶ *Designed Circular Doubly Linked List node memory topology.*
- ▶ *Verified Win32 SND_ASYNC non-blocking audio playback in C++.*
- ▶ *Outlined terminal menu state machine and user playback actions.*

Individual Contribution

Member	Role	Contribution
Bhuvi Singhal	Leader / Core	Architecture blueprint and CDLL cyclic pointer logic
Akshat Joshi	Algorithm Dev	Fisher-Yates shuffle logic and permutation validation
Akshank Chauhan	Data Structure	Custom Hash Set design and collision probing logic
Tansi Singh	Logic/Testing	LIFO undo stack design and playlist.txt file parser

Roadmap

1. *Phase-I: Architecture design, pointer layouts, and node planning*
2. *Phase-II: Core C++ implementation of CDLL, Hash Set, and Stack*
3. *Phase-III: Win32 audio binding, UI refinement, and leak auditing*

Expected Outcomes

- ▶ *A functional terminal-based audio engine playing local .wav files*
- ▶ *Mathematically proven zero-repeat shuffle via active hash tracking*
- ▶ *Zero heap memory leaks confirmed through destructor test logs*

References

1. *D.E. Knuth, The Art of Computer Programming, Vol. 2*
2. *Microsoft MSDN: Win32 Multimedia API Documentation*
3. *Spotify Engineering: "How to shuffle songs"*
4. *GeeksforGeeks and Cppreference (Memory Management)*